

MODULE-2

BCSE306 Artificial Intelligence

Dr.Priyanka.N

Assistant Professor Senior Grade-1

School of Computer Science and Engineering (SCOPE),

VIT Vellore,

Priyanka.n@vit.ac.in

Depth Limited Search

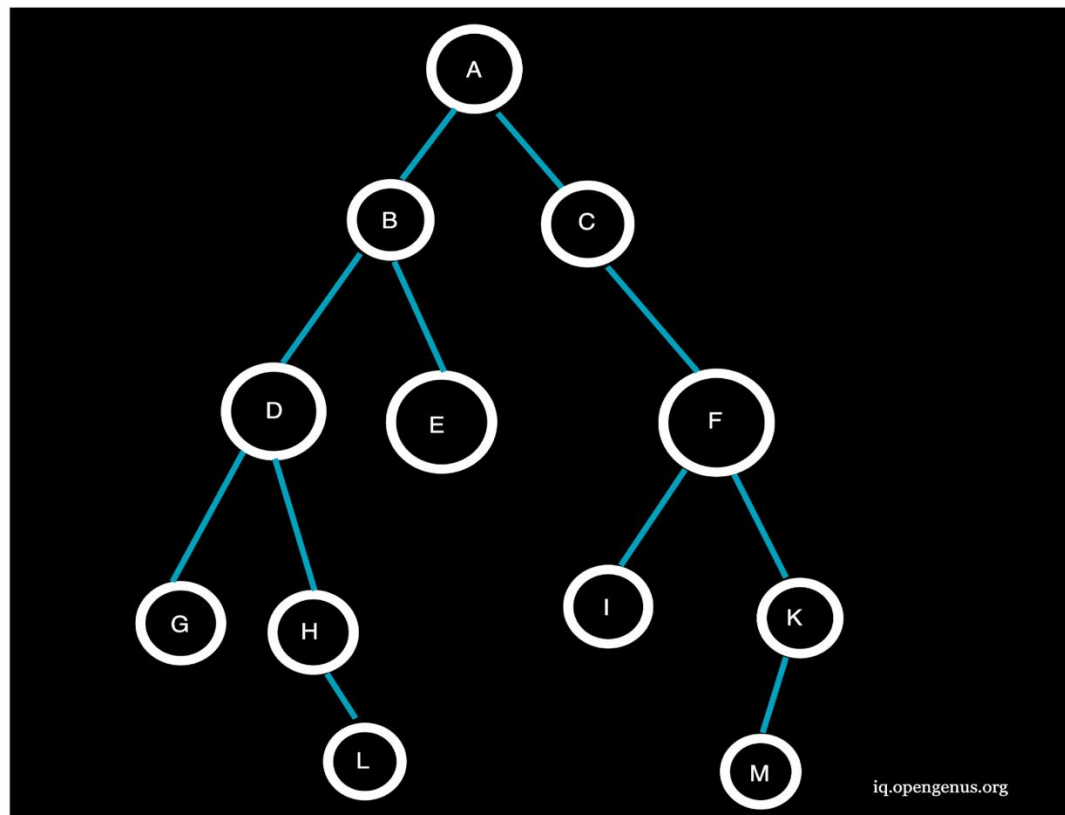
- Depth limited search is the **new search algorithm for uninformed search**.
- The **unbounded tree problem** happens to appear in the depth-first search algorithm, and it **can be fixed by imposing a boundary or a limit to the depth of the search domain**.
- We **denote this limit by 'l'**, and thus this **provides the solution to the infinite path problem** that originated earlier in the **DFS algorithm**.
- Thus, Depth limited search can be called an **extended and refined version of the DFS algorithm**.
- In a **nutshell**, we can say that **to avoid the infinite loop status** while executing the codes, and depth limited search algorithm is being executed into a finite set of depth called **depth limit**.

Algorithm

1. The **start node or node 1 is added** to the beginning of the stack.
2. Then **it is marked as visited**, and **if node 1 is not the goal node** in the search, **then we push second node 2 on top of the stack**.
3. Next, **we mark it as visited and check if node 2 is the goal node or not**.
4. If **node 2 is not found to be the goal node**, then we push node 4 on top of the stack.
5. Now we **search in the same depth limit and move along depth-wise** to check for the goal nodes.
6. If **Node 4 is also not found to be the goal node and depth limit is found to be reached**, then we **retrace back to nearest nodes that remain unvisited or unexplored**.
7. Then we **push them into the stack and mark them visited**.
8. We continue to **perform these steps in iterative ways unless the goal node is reached or until all nodes within depth limit** have been explored for the goal.

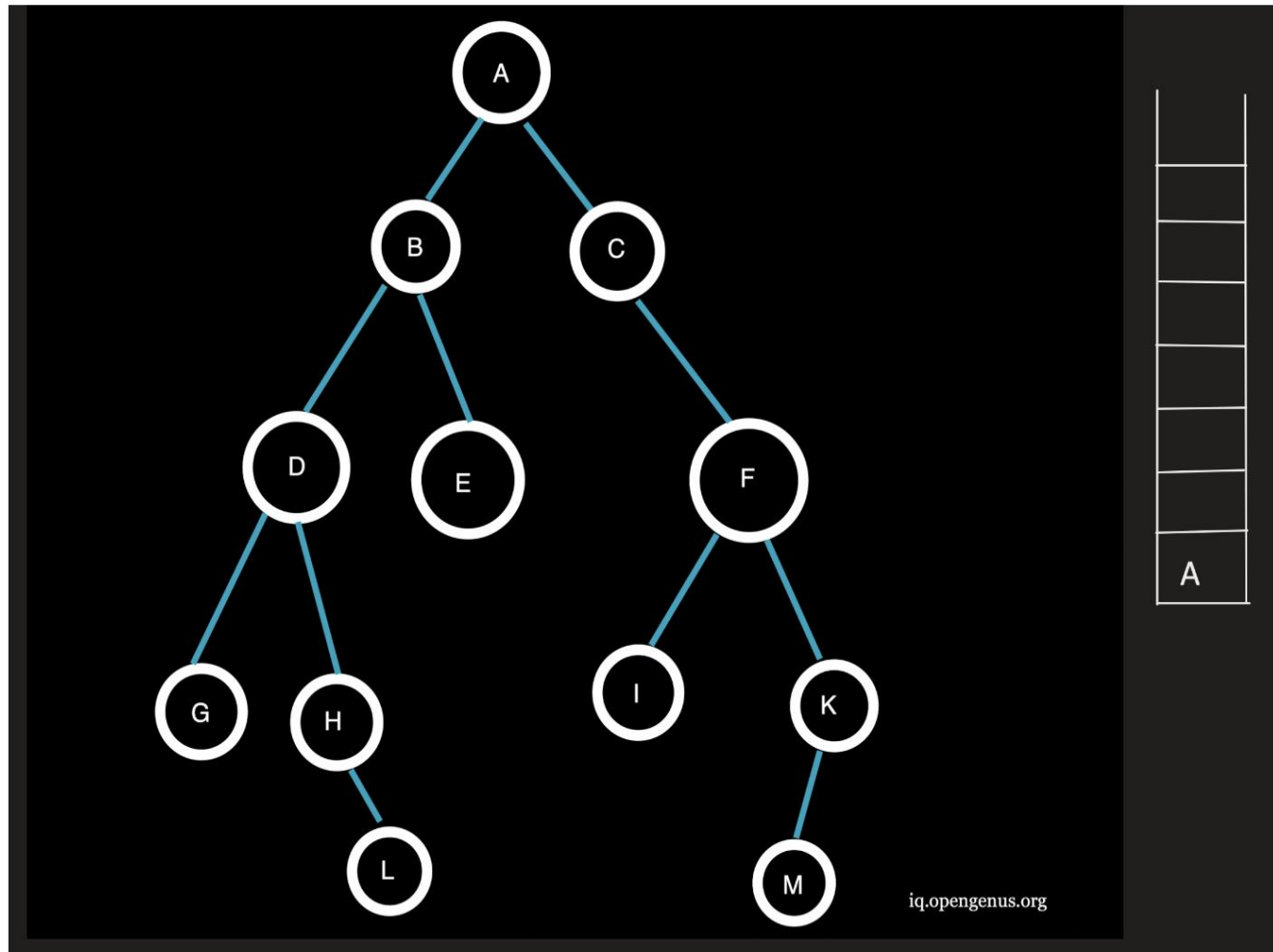
Example

Consider the given graph with Depth Limit(l)=2,
Target Node=H and the given source node=A



Example

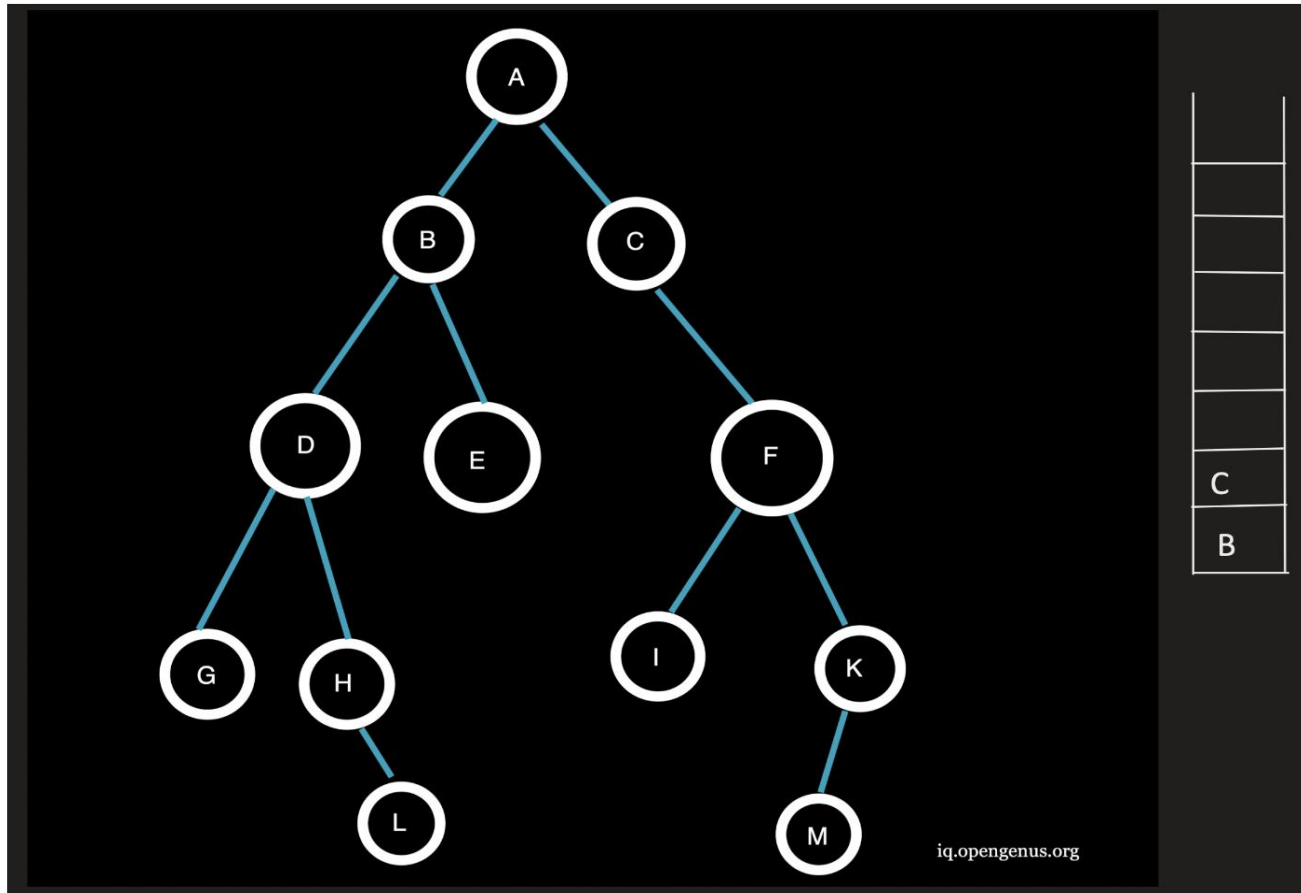
Step 1 :



Now, the first element of the source node is pushed onto the stack.

Example

Step 2 :

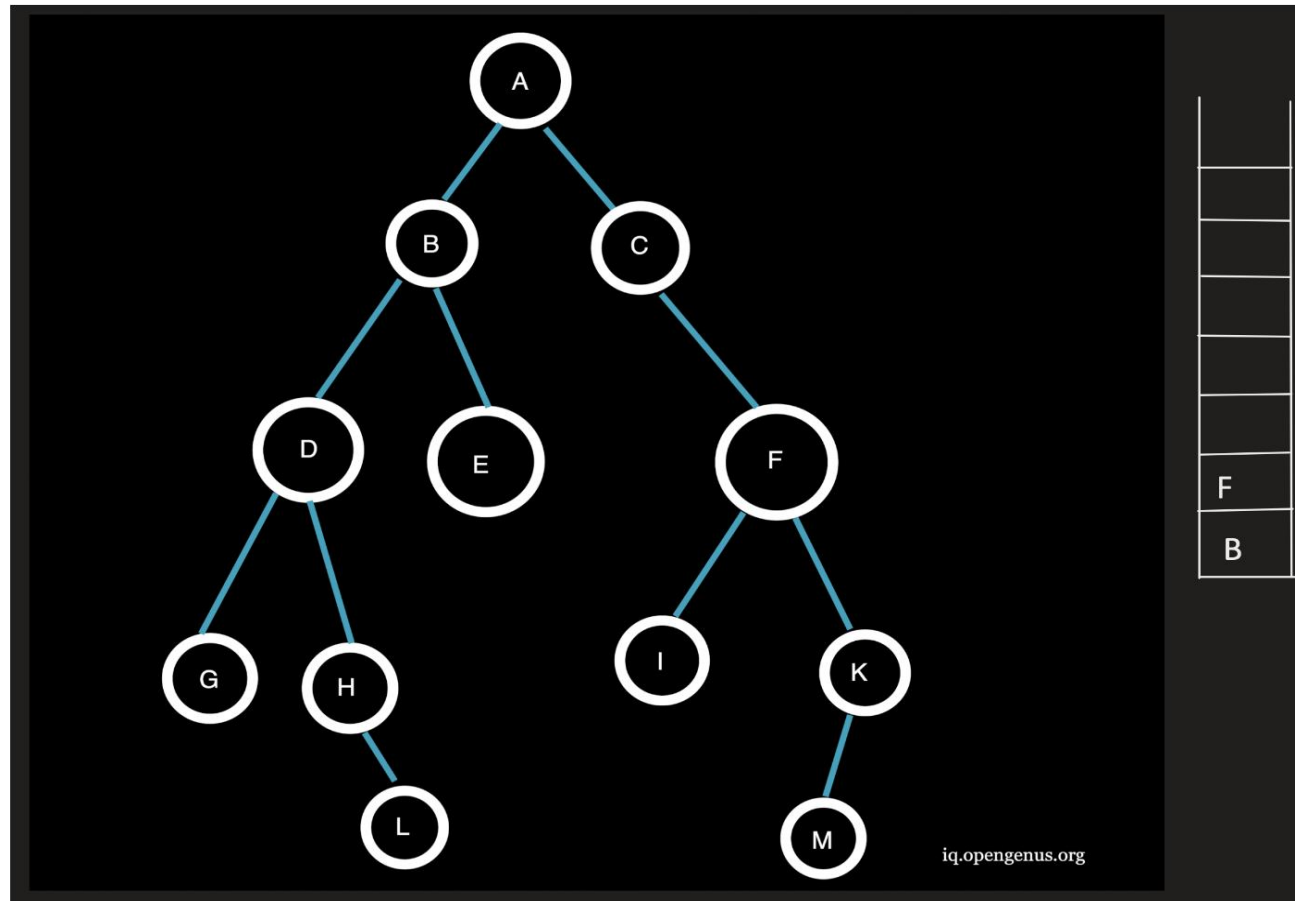


A being the top element is now popped from the stack and the neighbouring nodes B and C at depth=1(<1) of A are pushed onto the stack.

Traversal: A

Example

Step 3 :

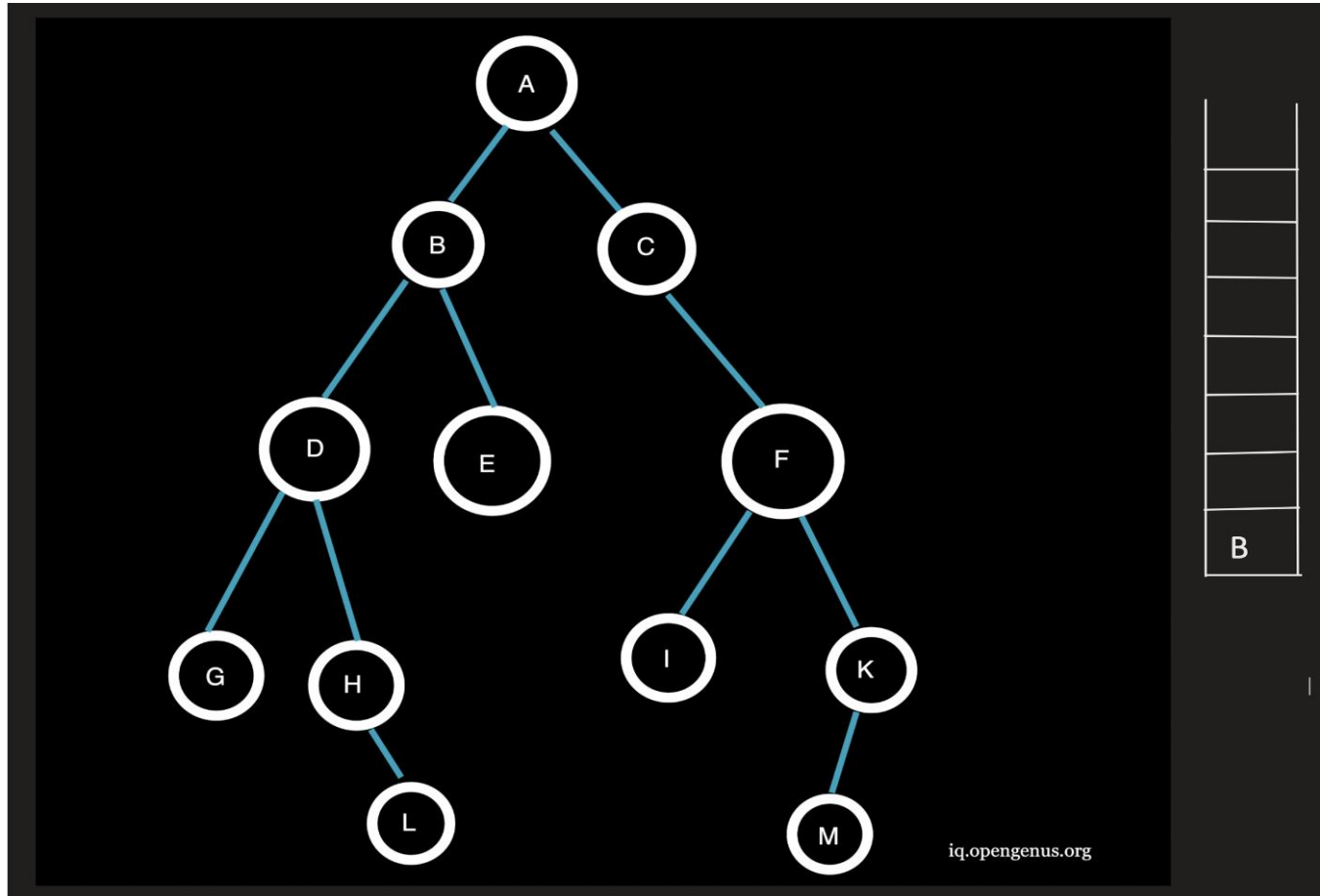


C being the topmost element is popped from the stack and the neighbouring node F at depth=2(=1) is pushed onto the stack.

Traversal: AC

Example

Step 4 :

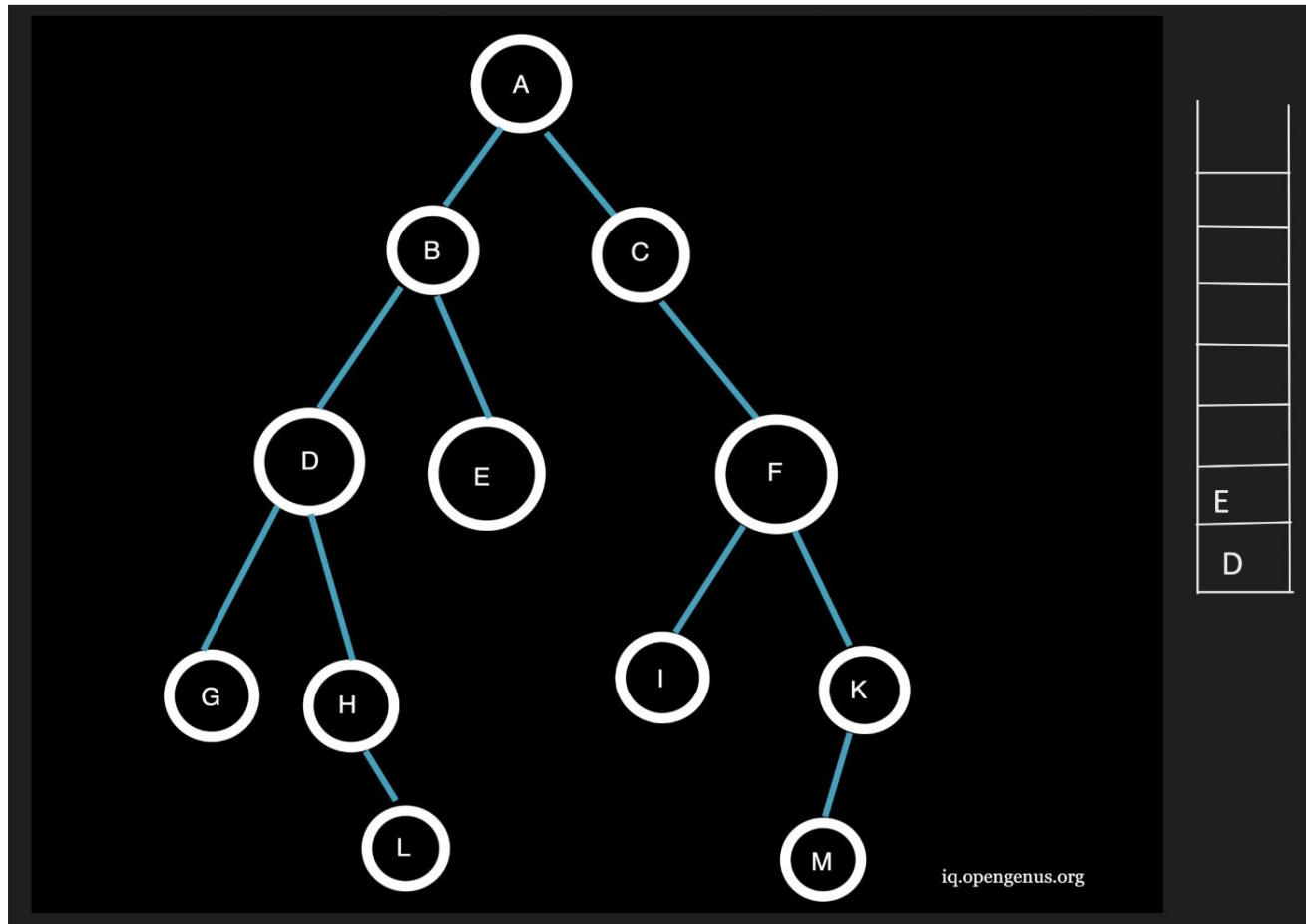


F being the topmost element is popped from the stack and the neighbouring nodes I and J at depth=3(>1) will not be pushed onto the stack.

Traversal: ACF

Example

Step 5 :

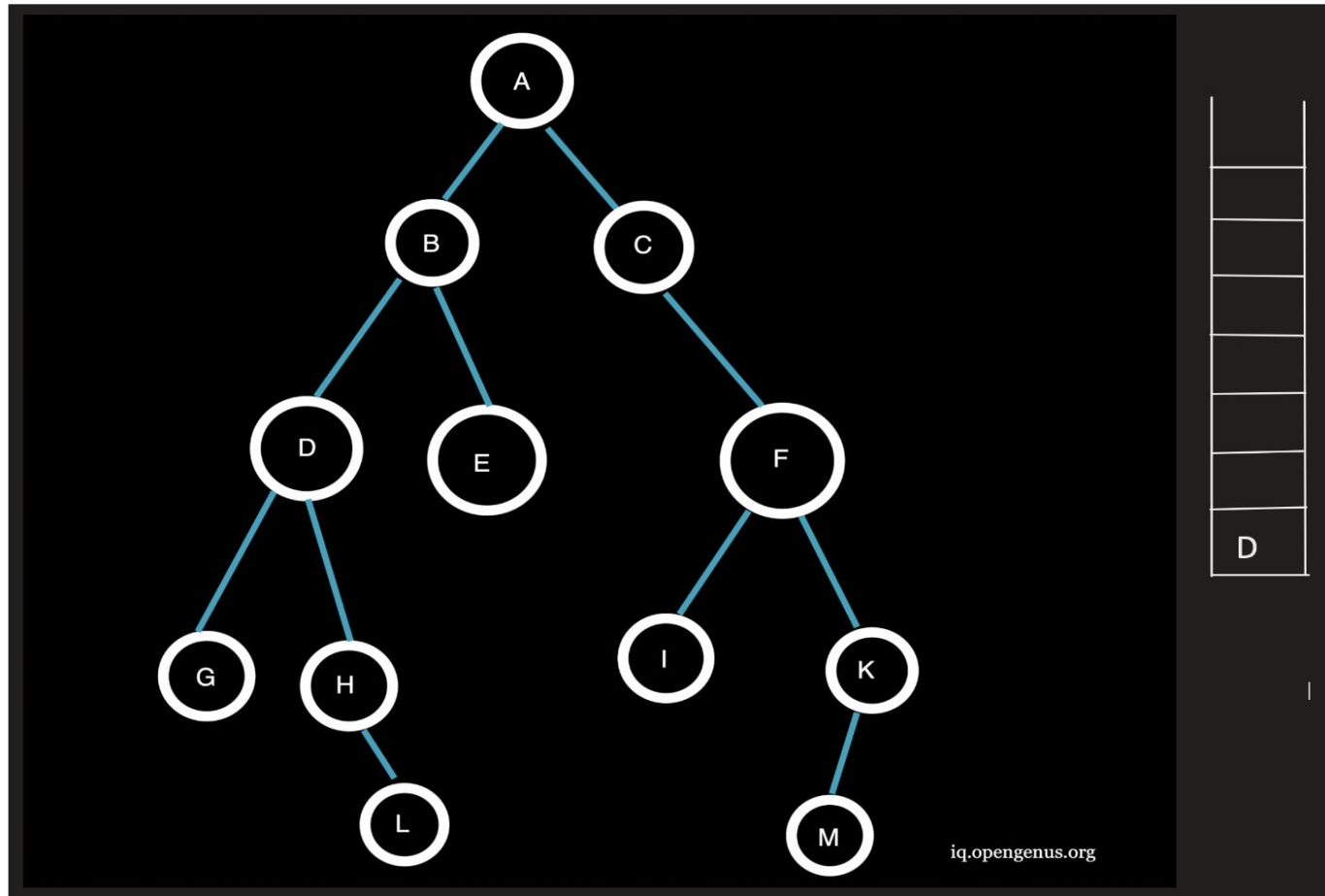


B being the topmost element is popped from the stack and the neighbouring nodes D and E at depth=2(=1) are pushed onto the stack.

Traversal: ACFB

Example

Step 6 :

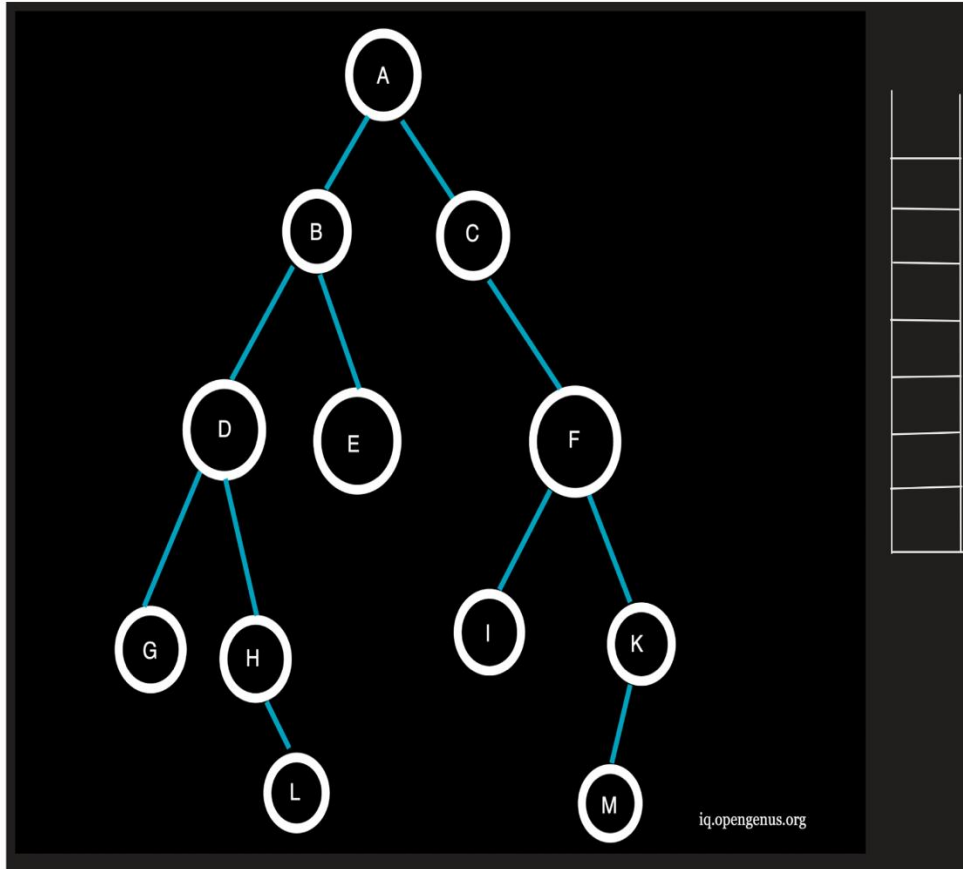


E being the topmost element is popped from the stack and since E has no neighbouring nodes, no nodes are pushed onto the stack.

Traversal: ACFBE

Example

Step 7 :



Since the stack is now empty, all nodes within the depth limit have been visited, but the target node H has not been reached.

D being the topmost element is popped from the stack and the neighbouring nodes G and H at depth=3(>1) will not be pushed onto the stack.

Traversal: ACFBED

Advantages

- Depth limited search **is better than DFS** and requires less time and memory space.
- DFS assures that the **solution will be found if it exists infinite time.**
- There are **applications of DLS in graph theory particularly similar to the DFS.**
- To combat the disadvantages of DFS, **we add a limit to the depth**, and our search strategy performs recursively down the search tree.

Disadvantage

- The **depth limit is compulsory for this algorithm** to execute.
- The **goal node may not exist in the depth limit set earlier**, which **will push the user to iterate further adding execution time.**
- The goal node will not be found **if it does not exist in the desired limit.**

Performance Measures

- **Completeness:** The DLS is an **incompleteness algorithm** in general except the case **when the goal node is the shallowest node**, and it is **beyond the depth limit, i.e. $l < d$** , and in this case, **we never reach the goal node**.
- **Optimality:** The DLS is a **non-optimal algorithm** since the depth that is chosen can be greater than d ($l > d$). Thus, DLS is not optimal if $l > d$
- **Time complexity is expressed as:** It is similar to the DFS, i.e. $O(b^l)$, where l is the set depth limit.
- **Space Complexity is expressed as:** It is similar to DFS. $O(b^l)$, where l is specified depth limit.

```

function DEPTH-LIMITED-SEARCH(problem, limit) returns a solution, or failure/cutoff
  return RECURSIVE-DLS(MAKE-NODE(problem.INITIAL-STATE), problem, limit)

function RECURSIVE-DLS (node , problem, limit) returns a solution, or failure/cutoff
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  else if limit = 0 then return cutoff
  else
    cutoff_occurred?  $\leftarrow$  false
    for each action in problem.ACTIONS(node.STATE) do
      child CHILD-NODE(problem , node, action)
      result RECURSIVE-DLS(child, problem, limit - 1)
      if result = cutoff then cutoff_occurred?  $\leftarrow$  true
      else if result  $\neq$  failure then return result
    if cutoff_occurred? then return cutoff else return failure

```

Figure 3.17 A recursive implementation of depth-limited tree search.

Comparison of DLS with BFS and DFS

S No.	Parameters	Depth First Search(DFS)	Depth Limited Search(DLS)	Breadth First Search(BFS)
1.	<i>Tree Traversal</i>	Depth-Wise	Depth-Wise	Level-Wise
2.	<i>Implementation</i>	Stack LIFO(Last-in First-out)	Stack LIFO(Last-in First-out)	Queue FIFO(First-in First-out)
3.	<i>Memory Required</i>	Lower	Lower	Higher
4.	<i>Backtracking</i>	Allowed	Allowed	Not Allowed
5.	<i>Infinite Loops</i>	Yes	No	No
6.	<i>Time Complexity</i>	$O(b^m)$ m=maximum depth of search tree	$O(b^l)$ l=specified depth limit	$O(b^d)$ d=depth of shallowest tree
7.	<i>Space Complexity</i>	$O(b^m)$	$O(b \times l)$	$O(b^d)$
8.	<i>Optimal</i>	No	No	Yes
9.	<i>Completeness</i>	No	Yes, when $d < l$ (depth-limit)	Yes
10.	<i>Applications</i>	1.Finding connected nodes or components 2.Used for detecting cycles	1.Solving Puzzles with only one solution 2.Topological sorting	1.Finding the shortest path between two nodes 2.Examining the bipartite graph